



Laboratorio 2: Sistemas Distribuidos

DiscoPass

Profesor: Jorge Díaz

Ayudantes de Lab: Felipe Marchant V. & Nelson Sepúlveda V.

Mayo 2025

1 Objetivos del Laboratorio

- Comprender y aplicar **tolerancia a fallos** en sistemas distribuidos.
- Implementar un **sistema de almacenamiento replicado**.
- Diseñar un **broker centralizado** con validación de identidades y gestión de consumidores.

2 Tecnologías

- El lenguaje de programación a utilizar es **Go**.
- Para la comunicación síncrona se utilizará **gRPC**.
- Para la definición de mensajes síncronos se usará **ProtocolBuffers**
- Para distribuir el programa se utilizará **Docker**

3 Introducción

La venta de entradas para eventos nocturnos enfrenta grandes desafíos durante períodos de alta demanda, especialmente cuando distintas discotecas publican fiestas, preventas o eventos especiales con cupos limitados.

En estos escenarios, una gran cantidad de usuarios puede intentar consultar, reservar o comprar entradas en tiempo real, por lo que el sistema debe ser capaz de procesar solicitudes de manera rápida, segura y sin pérdida de información.

En este laboratorio, se simulará un ecosistema distribuido de venta y validación de entradas para discotecas, compuesto por:

- **Productores (Discotecas)** que publican eventos y entradas disponibles.
- Un **Broker central** que valida, almacena y distribuye los eventos, además de coordinar las solicitudes de compra.
- Un **Sistema de almacenamiento distribuido** replicado en múltiples nodos, encargado de guardar eventos, compras, tickets y estados de pago.
- **Consumidores (Usuarios)** .
- Un **Servicio de pago simulado**, encargado de validar si una compra de entrada es aprobada o rechazada.

El sistema debe ser **tolerante a fallos**: la caída de un nodo de base de datos, de un consumidor o del servicio de pago no debe detener completamente la operación. Además, se debe evitar la sobreventa de entradas, la duplicación de tickets y la pérdida de información histórica.

4 Laboratorio

4.1 Contexto

En el mundo del entretenimiento nocturno, la venta de entradas para discotecas y eventos se ha convertido en un proceso altamente dinámico y competitivo. Durante fiestas masivas, preventas universitarias o eventos temáticos, cientos o miles de usuarios pueden conectarse simultáneamente para consultar disponibilidad, recibir notificaciones y comprar entradas antes de que se agoten los cupos.

En este escenario, las discotecas deben publicar eventos, actualizar stock, gestionar precios y mantener disponible la información en tiempo real. Al mismo tiempo, la plataforma debe validar las compras, evitar la duplicación de tickets.

4.2 Explicación

Para este laboratorio, deberán simular un sistema distribuido de venta y validación de entradas para cuatro discotecas ficticias:

- DataClub
- Dockers Night
- GoLounge
- GeorgieHouse

Cada una de estas discotecas corresponde a un **productor de entradas**, encargado de publicar eventos nocturnos con información como nombre del evento, fecha, precio, stock disponible y ubicación. Estas publicaciones serán enviadas constantemente hacia un **broker central**, el cual actuará como la plataforma principal de venta de entradas.

El broker debe validar la identidad de cada discoteca antes de aceptar sus publicaciones. Además, debe **almacenar y distribuir la información de los eventos** hacia un sistema de **base de datos**

distribuida, el cual funciona bajo un modelo de **replicación tipo DynamoDB**. De esta forma, los eventos, compras y tickets generados quedan respaldados en múltiples nodos.

Por otra parte, los **usuarios** podrán consultar los eventos disponibles y solicitar la compra de entradas. Cuando un usuario realiza una compra, el broker debe verificar que exista stock suficiente, consultar un **servicio de pago simulado** y, si la transacción es aprobada, generar un ticket único para el usuario. Este ticket debe quedar registrado en la base de datos distribuida para permitir su posterior recuperación o validación.

Durante la simulación, deberán además considerar **fallos inevitables**:

- Un nodo de base de datos que deja de responder.
- Usuarios que se desconectan durante el proceso de compra o consulta.
- Fallos temporales en el servicio de pago.
- Replicaciones incompletas que luego deben sincronizarse.

El reto será demostrar que, incluso bajo estas condiciones, el sistema se mantiene disponible, consistente bajo las reglas de DynamoDB (N, W, R), evita la sobreventa de entradas, registra correctamente las compras y permite recuperar la información histórica sin pérdida de datos.

4.3 Discotecas

Los Productores de Entradas corresponden a cuatro discotecas **DataClub**, **Dockers Night**, **GoLounge** y **GeorgieHouse**. Su misión es publicar eventos nocturnos en tiempo real y transmitir la información al Broker Central para su almacenamiento, validación y posterior gestión de compras por parte de los usuarios.

Cada productor simula una discoteca distinta, con sus propios eventos, precios, aforos y stock de entradas disponibles. Estos mensajes deben seguir una estructura uniforme y ser validados por el broker antes de ser aceptados en el sistema.

Funciones de los Productores:

- **Generación de Eventos:** Cada discoteca crea eventos nocturnos de distintas categorías, como electrónica, reggaetón, pop, techno, noche universitaria, fiesta temática, entre otras.
- **Envío al Broker:** Los eventos y entradas disponibles se envían al Broker Central mediante comunicación gRPC.
- **Identificación Segura:** Cada discoteca debe registrarse en el broker antes de operar. En caso contrario, sus mensajes serán rechazados.
- **Idempotencia:** Cada evento debe incluir un identificador único (`evento_id`) que permita al broker descartar publicaciones duplicadas.

Reglas de Generación de Eventos

Cada discoteca debe contar con un catálogo base de eventos o tipos de fiesta. Este catálogo será entregado por los ayudantes y tendrá diferentes nombres de eventos, comunas, stock, etc. A partir de dicho catálogo, los eventos se generarán de forma periódica en tiempo real, utilizando variaciones de precio a medida que el stock vaya disminuyendo, stock disponible y tipo de evento.

Las entradas serán publicadas por las mismas discotecas considerando un precio base y posibles variaciones según la demanda o el tipo de fiesta. La frecuencia de emisión debe ser continua, por ejemplo, una publicación de evento 30–40 segundos de intervalo.

Todos los eventos deben incluir un stock estrictamente mayor que cero, un precio válido y una fecha correspondiente al momento exacto de su publicación o actualización, basada en `time.Now()` en Go.

Estructura del Mensaje de Evento (ejemplo):

Listing 1: Ejemplo de Evento de Discoteca

```
{
  "evento_id": "uuidv4",
  "discoteca": "DataClub",
  "nombre_evento": "Noche Reggaeton Old School",
  "categoria": "Electronica",
  "comuna": "Huechuraba",
  "precio": 12000,
  "stock": 300,
  "fecha_evento": "2025-10-20T23:00:00",
  "fecha_publicacion": "2025-10-10T18:30:00"
}
```

4.4 Broker Central

El Broker Central es el núcleo del sistema. Su misión principal es recibir los eventos publicados por las discotecas, validarlos, almacenarlos de manera distribuida y coordinar las solicitudes de compra realizadas por los usuarios.

El broker actúa como punto de control y consistencia, asegurando que las reglas de replicación $N=3$, $W=2$, $R=2$ se cumplan, y gestionando la relación entre discotecas, usuarios, servicio de pago y base de datos distribuida.

Funciones del Broker Central:

- **Registro y Autenticación:** Mantiene un registro de discotecas, nodos DB, usuarios y servicio de pago activo. Rechaza entidades no autorizadas.
- **Recepción de Eventos:** Recibe todos los eventos enviados por las discotecas productoras, tales como fiestas, noches temáticas o actualizaciones de stock de entradas.
- **Validación de Eventos:** Verifica que cada evento tenga datos válidos, como identificador único, nombre del evento, discoteca, categoría, comuna, precio, stock disponible y fecha correspondiente.
- **Almacenamiento Distribuido:** Envía cada evento o compra confirmada a los 3 nodos de base de datos ($N=3$), esperando confirmación de al menos 2 nodos ($W=2$) para considerar exitosa la escritura.
- **Gestión de Compras:** Recibe solicitudes de compra de entradas por parte de los usuarios, verifica que exista stock disponible y coordina la validación del pago con el servicio externo correspondiente.
- **Generación de Tickets:** Si el pago es aprobado y existe stock disponible, genera un identificador único de ticket (`ticket_id`) y registra la compra en la base de datos distribuida.
- **Recuperación de Datos:** Ante consultas de usuarios o recuperación posterior a una desconexión, coordina lecturas en los nodos DB, aceptando la información solo si al menos 2 nodos coinciden ($R=2$).

- **Control de Stock:** Actualiza el stock disponible de cada evento luego de una compra exitosa, evitando la sobreventa de entradas.
- **Generación de Reporte Final:** Crea un archivo **Reporte.txt** con estadísticas de eventos procesados, compras realizadas, tickets generados, pagos rechazados y fallos ocurridos.

4.5 Banco USM

La Entidad Bancaria corresponde al organismo encargado de validar la disponibilidad de fondos del consumidor antes de autorizar una compra. Su función principal es verificar si el consumidor posee saldo suficiente en su cuenta bancaria o medio de pago asociado.

Cuando el consumidor realiza una compra, la solicitud de pago es enviada al **broker**, quien actúa como intermediario entre el sistema de compra y la entidad bancaria. Posteriormente, el banco valida la operación y responde si la transacción puede ser aceptada o rechazada.

Simulación de Validación: Para efectos de este laboratorio, el banco no requiere mantener una base de datos persistente de saldos. La validación se realizará de forma estocástica utilizando la librería `math/rand`. Por regla general, el banco deberá aprobar el 80% de las transacciones y rechazar el 20% restante por “fondos insuficientes”.

Regla por Medio de Pago: Si la solicitud de pago indica que el **medio de pago** es “crédito”, la probabilidad de aprobación del banco debe aumentar automáticamente a un 90%.

- **Responsabilidad principal:** validar la disponibilidad de dinero del consumidor mediante una respuesta probabilística.
- **Entrada:** solicitud de pago enviada a través del broker (incluye ID de usuario, monto y medio de pago).
- **Proceso:** generación de un resultado aleatorio basado en las probabilidades configuradas (80% general o 90% para crédito) y registro de la operación en logs.
- **Salida:** aprobación o rechazo de la compra.

El banco debe generar un registro (log) detallado por cada solicitud recibida, indicando el ID del usuario, el medio de pago utilizado, el monto de la transacción y el resultado final de la operación (Aprobado/Rechazado). En caso de aprobación, el broker podrá continuar con la generación del ticket; de lo contrario, la operación se dará por finalizada informando al usuario.

4.6 Nodos de Base de Datos Distribuida

Los Nodos de Base de Datos simulan el modelo de replicación eventual de DynamoDB. Existen 3 nodos en total (**DB1**, **DB2**, **DB3**) y cada operación de escritura o lectura debe cumplir con los parámetros de consistencia **N=3**, **W=2**, **R=2**.

Funciones de los Nodos DB:

- **Almacenamiento Replicado:** Guardan cada evento, compra y ticket recibido desde el broker.

- **Confirmación de Escritura:** Devuelven ACK al broker si logran almacenar correctamente la información recibida.
- **Replicación Eventual:** Si un nodo queda fuera de línea, debe resincronizarse con sus pares al volver a estar activo, recuperando eventos, compras y tickets pendientes.
- **Respuesta a Lecturas:** Devuelven información histórica al broker, como eventos publicados, compras realizadas, tickets generados y estado de pagos. La lectura solo es válida si al menos 2 nodos coinciden en los datos.
- **Respaldo de Stock:** Mantienen el registro actualizado del stock disponible de cada evento, permitiendo verificar compras y evitar inconsistencias.

Fallos Simulados:

- **Caída Temporal:** Un nodo puede dejar de responder, afectando temporalmente las operaciones de escritura o lectura.
- **Recuperación:** Al reintegrarse, debe recibir el backlog de datos pendientes desde los otros nodos para quedar nuevamente sincronizado.
- **Inconsistencia Temporal:** Un nodo puede quedar desactualizado respecto a compras o actualizaciones de stock, por lo que debe corregirse mediante replicación eventual.

4.7 Usuarios Compradores

Los Usuarios Compradores representan a las personas que ingresan a la plataforma para consultar eventos disponibles y comprar entradas para distintas discotecas. En este contexto los usuarios consultan directamente la cartelera disponible y seleccionan un evento para intentar realizar una compra. El sistema contará con dos entidades de usuarios independientes (Cliente A y Cliente B). Ambas representan clientes que acceden simultáneamente al sistema para consultar eventos y comprar entradas. Cada una de estas entidades operará como un proceso aislado.

Funciones de los Usuarios Compradores:

- **Registro Inicial:** Cada usuario se registra en el broker antes de comenzar a operar.
- **Consulta de Eventos Disponibles:** Los usuarios solicitan al broker la lista de eventos publicados por las discotecas.
- **Selección de Evento:** Cada usuario selecciona un evento disponible, el cual puede ser elegido de forma aleatoria durante la simulación.
- **Solicitud de Compra:** El usuario solicita la compra de una entrada para el evento seleccionado. Esta solicitud (vía gRPC) debe incluir obligatoriamente el `id_usuario` y el `medio_pago` extraído de su configuración.
- **Validación de Compra:** El broker verifica si existe stock disponible y coordina la validación del pago con el servicio de pago simulado.
- **Recepción de Ticket:** Si la compra es aprobada, el usuario recibe un `ticket_id` único asociado a su entrada.

- **Almacenamiento Local:** Cada usuario guarda sus compras confirmadas en un archivo CSV propio, por ejemplo `usuario_15.csv`.
- **Recuperación de Historial:** Si un usuario se desconecta, al volver puede solicitar al broker su historial de compras y tickets generados.

4.8 Fases de la Ejecución

El sistema deberá desarrollarse y ejecutarse siguiendo una serie de fases claramente definidas. Cada una de estas fases deberá poder observarse durante la ejecución del laboratorio.

1. Registro de Entidades

- Todas las discotecas, nodos de base de datos, usuarios y el servicio de pago deben registrarse en el broker antes de comenzar a operar.
- El broker debe validar identidades y rechazar mensajes provenientes de entidades no registradas.

2. Publicación de Eventos

- DataClub, Dockers Night, GoLounge y GeorgieHouse generan y envían sus eventos al broker mediante gRPC, de manera periódica.
- Cada discoteca debe utilizar su propio catálogo base de eventos y variar dinámicamente datos como precio, stock disponible y fecha del evento.
- Cada evento debe contener un identificador único, nombre del evento, discoteca, categoría, comuna, precio, stock y fecha.

3. Escritura Distribuida ($N=3$, $W=2$)

- El broker reenvía cada evento publicado a los 3 nodos de base de datos.
- La operación de escritura solo se considera exitosa si al menos 2 nodos ($W=2$) confirman el almacenamiento.
- Los nodos deben implementar replicación eventual para resincronizarse tras fallas.

4. Consulta y Compra de Entradas

- Los usuarios pueden consultar al broker la lista de eventos disponibles.
- Cada usuario puede solicitar la compra de una entrada para un evento específico.
- El broker debe verificar que el evento exista y que aún tenga stock disponible.
- Si existe stock, el broker debe consultar al servicio de pago simulado para aprobar o rechazar la compra.
- Si la compra es aprobada, el broker genera un `ticket_id` único, actualiza el stock y registra la compra en la base de datos distribuida.
- Cada usuario almacena sus compras confirmadas en un archivo CSV propio.

5. Simulación de Fallos

- Durante la ejecución, al menos un nodo de base de datos debe fallar temporalmente.
- Además, uno o más usuarios pueden desconectarse durante una consulta o proceso de compra.
- También se puede simular una falla temporal del servicio de pago.
- El sistema debe continuar funcionando siempre que se cumplan las condiciones mínimas de escritura y lectura distribuidas.

6. Lectura Distribuida ($R=2$)

- Los usuarios pueden consultar su histórico de compras y tickets al broker.
- El broker coordina la lectura en al menos 2 nodos ($R=2$) y devuelve el resultado solo si existe consenso entre los datos obtenidos.

7. Recuperación y Resincronización

- Los nodos de base de datos caídos deben resincronizarse al volver a estar activos.
- Los usuarios caídos deben recuperar su histórico de compras desde los nodos de lectura.
- Si una compra quedó pendiente por falla del servicio de pago, el broker debe dejar registro del estado correspondiente.

8. Generación de Reporte Final

- Al concluir, el broker debe generar un archivo **Reporte.txt** con un resumen detallado de la ejecución.
- El reporte debe incluir eventos publicados, compras aprobadas, compras rechazadas, tickets generados, fallos ocurridos y resultado de la resincronización.

4.9 Reporte Final

El archivo **Reporte.txt** debe ser generado por el broker al finalizar la ejecución. Este archivo debe incluir como mínimo las siguientes secciones:

1. Resumen de Discotecas

- Cantidad total de eventos enviados por cada discoteca.
- Cantidad de eventos aceptados por el broker.
- Cantidad de eventos rechazados por datos inválidos, stock nulo o identificadores duplicados.

2. Estado de Nodos de Base de Datos

- Estado final de cada nodo (activo, caído o recuperado).
- Número de escrituras exitosas y fallidas.
- Cantidad de eventos, compras y tickets almacenados en cada nodo.

3. Resumen de Compras y Tickets

- Cantidad total de solicitudes de compra realizadas por los usuarios.
- Cantidad de compras aprobadas.
- Cantidad de compras rechazadas por falta de stock.
- Cantidad de compras rechazadas por el servicio de pago.
- Cantidad total de tickets generados correctamente.
- Confirmación de que cada usuario generó su archivo CSV con las compras confirmadas.

4. Estado del Servicio de Pago

- Cantidad de pagos aprobados.
- Cantidad de pagos rechazados.
- Cantidad de solicitudes de pago fallidas o sin respuesta.

5. Fallos y Recuperaciones

- Listado de fallos ocurridos, como nodos caídos, usuarios desconectados o fallas temporales del servicio de pago.
- Resultados de la resincronización de nodos de base de datos.
- Recuperación de historial de usuarios desconectados.

6. Conclusión

- Indicación de si el sistema logró mantenerse disponible y consistente bajo las reglas especificadas.
- Indicación de si el sistema evitó la sobreventa de entradas y la duplicación de tickets.

4.10 Categorías Disponibles

Los eventos generados por las discotecas deberán pertenecer exclusivamente a alguna de las siguientes categorías:

- Electrónica
- Reggaetón
- Pop
- Techno
- House
- Urbana
- Latina
- Noche Universitaria
- Fiesta Temática
- Retro

- Open Bar
- VIP

El broker deberá validar que todos los eventos publicados pertenezcan a una de estas categorías. Eventos con categorías no reconocidas deben ser descartados. Además, cada evento debe contar con precio válido, stock mayor a cero, fecha correcta e identificador único.

5 Uso de Docker

Todos los procesos del laboratorio deben ejecutarse en contenedores Docker dentro de las máquinas virtuales. Todas las entidades deben estar aisladas en contenedores separados.

Ejemplo de distribución de las entidades en las máquinas virtuales:

- MV1: Broker
- MV2: Productores / DB3
- MV3: Consumidores / DB2
- MV4: Banco / DB1

6 Restricciones

Las librerías de Golang permitidas son:

- `fmt`
- `log`
- `errors`
- `os`
- `io`
- `bufio`
- `time`
- `sync`
- `math/rand`
- `encoding/csv`
- `encoding/json`
- `net`
- `context`
- `flag`
- `strconv`
- `strings`
- `grpc`

Además de las necesarias para el funcionamiento de gRPC y Protocol Buffers. Todo uso de librerías externas que no se han mencionado en el enunciado debe ser consultado con los ayudantes.

7 Consideraciones

- **Comunicación:**

- Todas las comunicaciones entre entidades deben realizarse estrictamente mediante **gRPC** y **Protocol Buffers**.
- No está permitido el uso de colas de mensajes externas como RabbitMQ, Kafka, NATS, etc.

- **Broker Central:**

- El broker es el único punto autorizado para interactuar con productores, entidad bancaria, nodos de base de datos y consumidores.
- El broker debe validar la identidad de cada entidad antes de aceptar conexiones o mensajes.
- Eventos con identificadores duplicados deben ser descartados para garantizar idempotencia.

- **Nodos de Base de Datos:**

- El sistema debe respetar los parámetros de consistencia: **N=3, W=2, R=2**.
- Si un nodo falla, las escrituras deben continuar funcionando mientras se cumpla $W = 2$.
- Si un nodo se reincorpora, debe resincronizarse automáticamente con sus pares.
- Lecturas históricas deben validar que al menos $R = 2$ nodos entreguen resultados coincidentes.
- Lecturas históricas solo deben usarse cuando un consumidor se reintegre luego de haberse caído.

- **Consumidores:**

- Los usuarios deben registrarse en el broker antes de consultar eventos o solicitar compras.
- Una solicitud de compra solo debe ser procesada si el evento existe, tiene stock disponible y el pago es aprobado por el servicio de pago simulado.
- Si una compra es exitosa, el usuario debe recibir un identificador único de ticket (`ticket_id`) y guardar la información en un archivo CSV propio.
- Los usuarios no deben generar tickets por cuenta propia; la generación del ticket es responsabilidad exclusiva del broker.
- Si un usuario se desconecta durante una consulta o compra, al reintegrarse debe poder recuperar su historial de compras y tickets desde el broker.
- El sistema debe evitar compras duplicadas de un mismo usuario para el mismo evento.
- Los usuarios deben recibir correctamente respuestas de rechazo, por ejemplo, compra rechazada por falta de stock, pago rechazado o evento no disponible.

- **Archivos de Configuración:**

- Los archivos necesarios serán entregados por los ayudantes. A la hora de evaluar se usará una configuración distinta así que nada debe estar “hardcodeado”.

- Los eventos por los productores deben incluir exclusivamente las categorías listadas en el enunciado.
- Los productores deben generar eventos únicamente a partir de categorías válidas, las que están listadas en el enunciado.

- **Contenedores Docker:**

- Cada entidad debe ejecutarse en un contenedor Docker independiente.
- Se debe incluir un `Makefile` con atajos como:
 - * `make docker-VM1`
 - * `make docker-VM2`
 - * `make docker-VM3`
 - * `make docker-VM4`

- **Logs y Reporte:**

- Todas las entidades deben generar logs claros sobre su ejecución (conexiones, mensajes recibidos, fallos simulados).
- El broker debe generar el archivo `Reporte.txt` al finalizar, con el detalle solicitado en el enunciado.

- **Consultas y Soporte:**

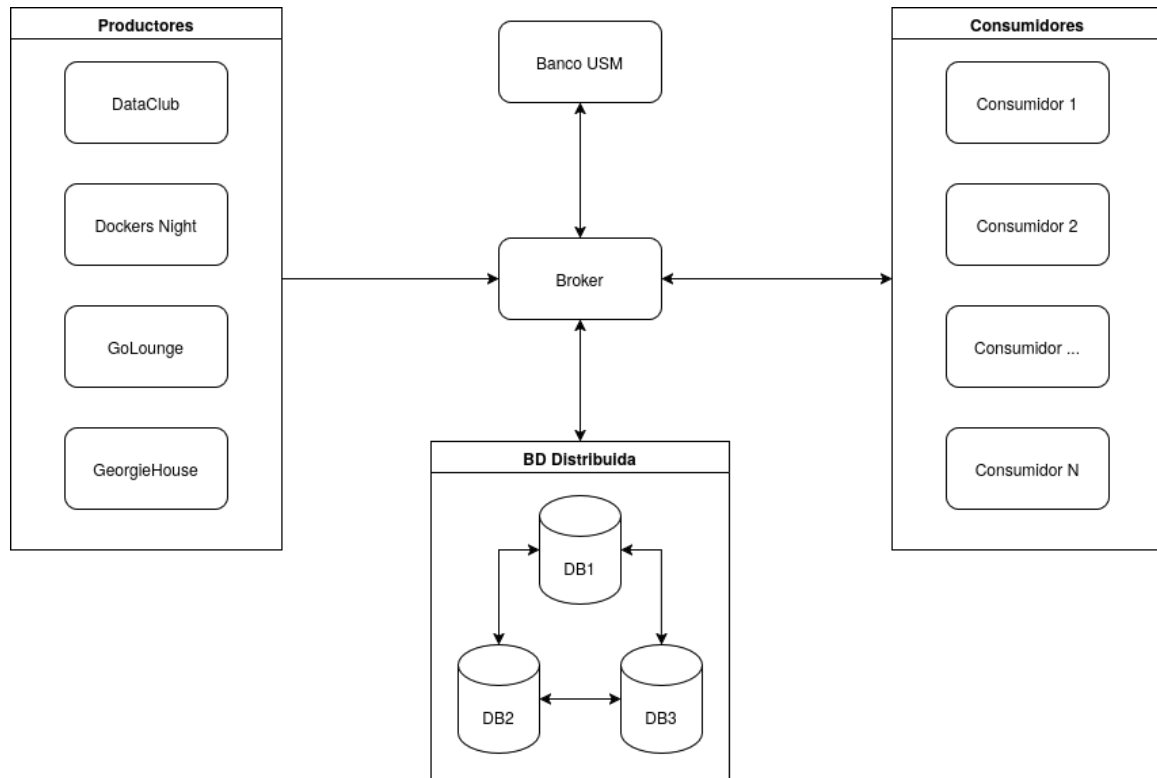
- Se realizará una ayudantía para detallar el enunciado y resolver dudas generales.
- Las consultas deben realizarse en el foro habilitado en Aula o al correo de los ayudantes:
 - * `felipe.marchantv@usm.cl`
 - * `nelson.sepulveda @usm.cl`
- No se responderán consultas enviadas con menos de **48 horas antes de la fecha de entrega original**. En caso de extensión de plazo, para esta regla se considerará la fecha original de entrega.

8 Reglas de Entrega

- La tarea se entrega en grupos de 2 a 3 personas previamente asignados en aula.
- La fecha de entrega es el día **10 de junio 23:59 hrs.**
- La tarea se revisará en las máquinas virtuales, por lo que los archivos necesarios para la correcta ejecución de esta deben estar en ellas. Recuerde que el código debe estar indentado, comentado, sin warnings y sin errores.
- Además de los códigos en las máquinas virtuales y github, deberán subir un archivo comprimido que contenga todos los códigos desarrollados en carpetas separadas por entidad en formato **.zip** con el nombre **GrupoXX-LabY.zip**. Donde XX es el número de su grupo e Y es el número del laboratorio. Ejemplo: *Grupo07-Lab2.zip*.
- Debe dejar un **MAKEFILE** o similar en cada máquina virtual asignada a su grupo para la ejecución de cada entidad.
- Debe dejar un **README** en la entrega asignada a su grupo con nombre y rol de cada integrante, además de la información necesaria para ejecutar los archivos.
- El uso de **Docker** es **obligatorio**. Teniendo nota 0 aquellas implementaciones que no hagan uso de Docker.
- Las faltas de README y/o copias serán evaluadas con nota 0.
- Todos los descuentos y aclaraciones necesarias están disponibles en el documento “**Reglas de Laboratorio**” disponible en el Aula virtual. Favor revisarlo con prontitud para evitar inconvenientes.

9 Anexos

9.1 Entidades



9.2 Diagramas de Secuencia

